

Real-Time Hand Sign Recognition Using MediaPipe and OpenCV

Course: Computer Vision Lab Project

Author: Chaitanya

Date: September 2026

1. Why I Chose This Project

When picking a topic for our computer vision course, I wanted to build something interactive that I could actually test with my laptop webcam in real time, rather than just running static image filters. Hand gesture recognition felt like a cool idea because we use hand gestures every day, and teaching a computer to understand them without any physical controllers or specialized sensors seemed like an interesting challenge.

Initially, I looked into the classic approach taught in class—using Haar Cascades or training a custom CNN. But training a deep neural network requires gathering thousands of annotated pictures of hands in different lighting and poses, plus my laptop doesn't have an external GPU to train large models. That's when I found Google's MediaPipe framework. It has a pre-trained hand landmark detector that tracks 21 distinct 3D points on a hand.

My goal was to take those raw landmark coordinates and write my own classification logic to reliably detect everyday gestures (like thumbs up, peace, pointing, rock-on, etc.) at a solid frame rate on a normal laptop CPU.

2. What I Wanted to Accomplish

I kept my objectives realistic and practical for this submission:

- Get a smooth webcam feed running with minimal delay.
 - Track 21 hand joints reliably using MediaPipe Hands.
 - Figure out a simple geometric way to check whether individual fingers are open or closed.
 - Correctly classify around 10 to 12 popular hand gestures based on finger states.
 - Handle tricky cases like distinguishing between "thumbs up" and "thumbs down", or detecting an "OK" pinch gesture.
 - Add an on-screen display (HUD) showing the detected gesture, confidence, and real-time FPS.
-

3. How the System Works

The entire pipeline runs frame-by-frame through a loop in OpenCV:

[Laptop Webcam]



1. Capture frame & flip horizontally (so it behaves like a mirror)



2. Pass RGB frame to MediaPipe Hands
 -
 - ▼
3. Extract 21 landmark points (x, y, z coordinates per hand)
 -
 - ▼
4. Determine state of each finger (extended vs. curled)
 -
 - ▼
5. Classify gesture using geometric rules
 -
 - ▼
6. Draw joints, bounding box, gesture name, and FPS onto screen

The 21 Landmarks

MediaPipe numbers the landmarks from 0 to 20:

- Landmark 0 is the wrist.
- 1 to 4 cover the thumb (from base to tip).
- 5 to 8 are the index finger.
- 9 to 12 are the middle finger.
- 13 to 16 are the ring finger.
- 17 to 20 are the pinky.

Each point has an x and y normalized between 0.0 and 1.0 based on frame width and height.

4. My Approach to Gesture Classification

Rather than throwing another machine learning model on top of the landmarks, I decided to use a rule-based geometric approach. It runs fast, uses practically zero extra memory, and when something doesn't work, it is easy to debug because you can inspect the coordinates directly.

Checking if Fingers are Open or Closed

For the index, middle, ring, and pinky fingers, checking whether a finger is extended is relatively simple:

- In screen coordinates, $y = 0$ is the top of the image and $y = 1$ is the bottom.
- If a finger is pointing up, its **TIP** will have a smaller y-value than its **PIP** (the middle knuckle joint).
- So if $\text{tip.y} < \text{pip.y}$, the finger is extended. Otherwise, it is curled into the palm.

The Problem with the Thumb

The thumb doesn't move vertically like other fingers; it folds across the palm horizontally. Checking y-values made thumb detection inaccurate.

To fix this, I had to:

1. Check the handedness (Left vs. Right hand) provided by MediaPipe.
2. Compare the x-coordinates: for a right hand facing the camera, an extended thumb sticks out to the left ($\text{tip.x} < \text{ip.x}$). For a left hand, it sticks out to the right ($\text{tip.x} > \text{ip.x}$).

3. For Thumbs Up vs. Thumbs Down, I added an extra check: for Thumbs Up, the thumb tip must be significantly higher than the knuckle (MCP). For Thumbs Down, the tip is positioned below the wrist.

Special Handling for the "OK" Sign

For an "OK" gesture, the index finger and thumb touch to form a circle while the other three fingers remain open. Here, checking up/down states wasn't enough. I calculated the Euclidean distance between the thumb tip and index tip:

$$\text{distance} = \sqrt{(x_{\text{thumb}} - x_{\text{index}})^2 + (y_{\text{thumb}} - y_{\text{index}})^2}$$

Through testing in front of the camera, a normalized distance threshold of 0.05 proved to be the sweet spot for detecting when the two tips pinch together.

5. Gestures Detected

Here are the 12 gestures currently supported:

Gesture Name	Hand Pose / Finger Combination
Fist	All 5 fingers curled
Open Hand	All 5 fingers extended
Peace / Victory	Index and middle fingers extended
Thumbs Up	Only thumb extended, pointing upwards
Thumbs Down	Only thumb extended, pointing downwards
Pointing	Only index finger extended
OK Sign	Thumb and index tips pinching together, other 3 fingers open
Rock On	Index and pinky extended (horns)
Call Me	Thumb and pinky extended
Spider-Man	Thumb, index, and pinky extended
Three	Index, middle, and ring extended
Four	All fingers extended except thumb

6. Development Struggles & What I Learned

Building this wasn't completely smooth sailing—here were the main hurdles I ran into:

1. The OpenCV 5.0 Surprise:

At the start, pip pulled opencv-python 5.0, which had deprecated and removed CascadeClassifier from the root namespace, breaking my initial face/hand detection attempts. Moving to MediaPipe ended up being a

much better decision anyway, and pinning `opencv-python<5` resolved compatibility.

2. Mirror Effect Confusion:

When I first tested the webcam feed, moving my hand to the left moved the on-screen hand to the right. It felt completely unnatural. Adding `cv2.flip(frame, 1)` right after reading from the webcam made it feel like a real mirror.

3. Inconsistent Lighting:

In dim room lighting or when sitting with a window directly behind me, MediaPipe occasionally flickered or lost landmark tracking. Raising `min_detection_confidence` to `0.6` cleaned up the jitter, but having decent ambient lighting is still important.

4. Finger Overlap at Odd Angles:

When turning my hand sideways (perpendicular to the webcam), fingers occlude one another and the 2D y-coordinate comparisons can misfire. The system works best when the palm is generally facing the camera.

7. Results & Performance

I tested the application on a standard Windows laptop running an Intel i5 processor with integrated graphics:

- **Frame Rate:** Runs consistently at **26 to 30 FPS**.
- **Latency:** Reaction feels instantaneous—detection takes under 40 milliseconds per frame.
- **Accuracy:** Everyday distinct gestures (Fist, Open Hand, Peace, Pointing, Rock On) have roughly a 95% success rate under regular indoor lighting. Gestures requiring precise positioning (OK sign, Thumbs Down) require deliberate hand placement but work reliably once adjusted.

8. What Could Be Added Next

If I expand this project later on, here are the top things I would want to tackle:

- **Gesture-controlled PC actions:** Mapping gestures to system shortcuts (e.g., thumbs up to raise volume, open hand to pause media, pointing to move the mouse cursor).
- **Dynamic gestures:** Detecting movement patterns over consecutive frames (like waving back and forth or swiping left/right).
- **American Sign Language (ASL):** Training a lightweight classifier like an SVM on top of the 21 landmarks to recognize the full A-Z alphabet.

9. Conclusion

This project showed me that you don't always need massive compute power or complex neural network architectures to solve practical computer vision problems. By leveraging a high-quality pre-trained landmark extractor like MediaPipe and pairing it with geometric rules in Python, I was able to build a functional, responsive hand gesture recognizer that runs smoothly on everyday hardware. It was a great hands-on way to understand spatial coordinates, real-time video pipelines, and user interaction in computer vision.

References

1. Zhang, F., et al., "MediaPipe Hands: On-device Real-time Hand Tracking", arXiv:2006.10214, 2020.
2. MediaPipe Solutions Documentation, Google Developers (<https://mediapipe.dev>).
3. Bradski, G., "The OpenCV Library", Dr. Dobb's Journal of Software Tools, 2000.
4. Rautaray, S. S., & Agrawal, A., "Vision based hand gesture recognition for human computer interaction: a survey", Artificial Intelligence Review, 2015.